

Queue (All Questions)

(Question # 01)

class Queue

{

private;

int *ary;
int front;
int rear;
int cap;
int cnt;

// Data members
// first element
// last element
// tot capacity

public:

Queue(int s)
{

cap = s;
ary = new int[cap];
front = 0;
rear = -1;
cnt = 0;

// Constructors

// Initialization

~Queue()

// destructor

{
delete [] ary;
}

bool isep() { return cnt == 0; } // if no entry

bool isfull() { return cnt == cap; } // if full

```
void enqueue(int x)
```

```
{
    if (isfull())
```

```
{
    return;
```

```
}
    rear = (rear + 1) % cap;
```

```
    arr[rear] = x;
```

```
    cnt++;
```

```
}
```

// if cap is full

// 0% suppose 3 = 1
 rear = 0
 → // arr[0] = 10 (sup)

→ cnt

```
void dequeue()
```

```
{
```

```
    if (isempty())
```

```
{
    return;
```

// if nothing (cnt == 0)

```
    front = (front + 1) % cap;
```

```
    cnt--;
```

```
}
```

1 % 4 = 1

→ cnt = 0

```
void display()
```

```
{
```

```
    if (isempty())
```

```
{
    return;
```

```
    for (int i = 0; i < cnt; i++)
```

let f = 0

```
    {
        int ind = (front + i) % cap;
```

```
        arr[ind];
```

```
    }
```

0 % 2 = 0

ind = 0

then

1 % 2 = 1

then

2 % 2 = 0 < cnt

Question # 02

Explained in basic operations of Queue

DRY_RUN

Suppose: Use queue 5

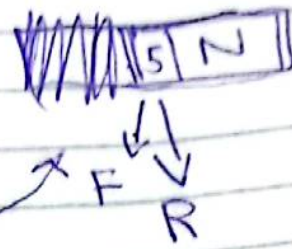
Node* nN = new Node(5);

if (—) ✓
{

F = R = NN;

return;

}



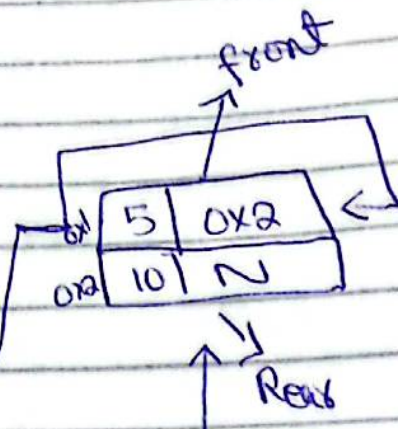
Now enqueue 10:

N* NN = new N(10);

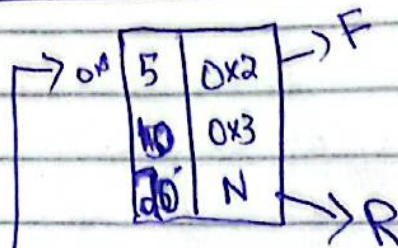
if (—) ✗

rear->next = NN;

rear = NN;



Now 20:



Now dequeue:

if (—) ✗

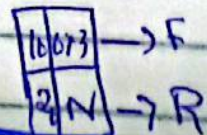
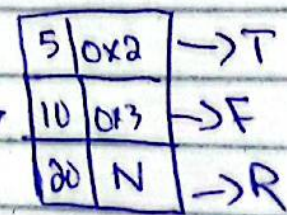
N* t = F;

F = F->Next;

if (F == Null) ✗

delete temp s.

So



Question # 03

```
class PriorityQueue
```

```
{  
    Private: any  
             int data;  
             int priority;  
             int cap;  
             int size;
```

// Data members

```
    Public:
```

```
    PriorityQueue(int c)
```

// constructor

```
{
```

```
    cap = c;
```

```
    data data = new int[cap];
```

```
    priority = new int[cap];
```

```
    size = 0;
```

```
}
```

// initialize

```
~PriorityQueue()
```

// destructor

```
{
```

```
    delete [] any;
```

```
    delete [] priority;
```

```
}
```

```
void enqueue(int v, int p)
```

```
{
```

```
    if (size == cap) { return; }
```

// full

```
    any[size] = val;
```

```
    priority[size] = p;
```

// insertions

```
    size++;
```

// increment size

```
}
```

void dequeue()

```
if (size == 0) { return; }
```

// no element

```
int hp = 0;
```

```
for (int i = 1; i < size; i++)
```

```
{ if (priority[i] > priority[hp])
```

```
{ hp = i;
```

// finding element
with highest
priority
index found here

```
}  
for (int i = hp; i < size - 1; i++)
```

```
{ arr[i] = arr[i+1];
```

```
priority = priority[i+1];
```

// Shifting 1
step before
to maintain
linearity

```
}  
size--;
```

// decrement size

void front()

```
if (size == 0) { return; }
```

// if no element

```
int hp = 0;
```

```
for (int i = 1; i < size; i++)
```

```
{ if (priority[i] > priority[hp])
```

```
{ hp = i; }
```

// finding element
with highest
priority

Question # 04

// same codes for Stack & Queue

(Dry-Run)

K=2

Suppose,

void reverse (Queue q, int k)

{
if (k <= 0 || k > q.size()) {

Stack S;

1	-
2	-
3	-
4	N

o) for (int i=0; i<k; i++)
{
s.push(q.dequeue());

2	-
1	-

o) while (!s.empty())

{
q.enqueue(s.pop());

3	-
4	-
2	-
1	-

o) int remaining = q.size() - k;

4-2=2

o) for (int i=0; i<rem; i++)

{
q.enqueue(q.dequeue());

3	-
4	-
2	-
1	-

Question # 05

// same code of Stack & Queue

(DRY RUN)

bool ispalindrome(Queue & q)

{
if (q.empty())

Suppose

1	
2	
1	

Stack S;

Queue T;

int qs = q.size(); qs = 3

for (int i = 0; i < qs; i++) ✓

{
int val = q.dequeue();

val = 1

S.push(val);

S → [1]

T.enqueue(val);

T → [1]

q.enqueue(val);

}

2	
1	
1	

for (—) ✓

{
int v = q.de();

val = 2

S.push(val);

T.enqueue(val);

q.enqueue(val);

}

1	
1	

2	
1	

1	
2	

1	
1	
2	

o) again $foo() \rightarrow \checkmark$ $0 < 3$

int $v = q.deg()$ $v = 1$

$s.push(v)$;

1	
2	
1	

$q.enqueue(v)$;

1
2
1

$q.enqueue(v)$;

1
2
1

o) bool $ispan = true$;

while ($!T.isep()$) $\checkmark$

{
if ($T.deg() \neq s.pop()$) $\times$

o) $wh() \checkmark$

{

if () $\times$

o) $wh() \checkmark$

{

if () $\times$

o) $wh() \times$

return $ispan$